



Numerical Optimization

Winter semester 2025/26

Exercise sheet 7 for Numerical Optimization

Manuel Caipo

Degree program: M.Sc. Computational Science Engineering
Enrollment number: 30783212

Instructors: Dr. Michael Lehn, Tobias Born

02. December 2025 - 89081 Ulm

Chapter 1

Solutions Exercise sheet 7

Method	Type of Derivatives Used	Search Direction d_k	Rule for Choosing Step Size α_k	Advantages / Disadvantages
Newton–Raphson	Gradient + Hessian (first and second derivatives)	$d_k = -H^{-1}\nabla f$	Line search such as Armijo or Wolfe	+ Very fast close to the minimum (quadratic convergence) – Needs Hessian; may fail if Hessian is not positive definite
Gauss–Newton	Jacobian (derivatives of residuals)	$d_k = -(J^T J)^{-1} J^T r$	Line search such as Armijo or Wolfe	+ Cheaper than full Newton; ideal for least-squares problems – Fails when $J^T J$ is nearly singular
Levenberg–Marquardt	Jacobian	$(J^T J + \lambda I)d_k = -J^T r$	Trust-region style update; often $\alpha_k = 1$	+ Very robust; transitions smoothly between gradient descent and GaussNewton – Requires tuning the parameter λ ; slower than GaussNewton
Linear Conjugate Gradient	Gradient	$d_k = -g_k + \beta_k d_{k-1}$	Exact step size for quadratic functions	+ Good for large linear systems; uses little memory – Only guaranteed for quadratic or linear problems
Nonlinear Conjugate Gradient (FletcherReeves, PolakRibiere, HestenesStiefel)	Gradient	$d_k = -g_k + \beta_k d_{k-1}$	Must use Wolfe conditions for convergence	+ Uses previous directions; low memory requirement – May lose descent property; needs Wolfe + Zoutendijk conditions
Hooke–Jeeves (Direct Search)	No derivatives required	Alternates exploratory moves and pattern moves	Reduces step size when no improvement is found	+ Works with noisy functions; does not require gradients – Slow; limited theoretical guarantees
Nelder–Mead Simplex	No derivatives required	Moves a simplex by reflection, expansion, contraction, shrink	Determined by simplex operations (not a classical line search)	+ Intuitive; works well in low dimensions – No general convergence guarantees; poor in high dimension

Table 1.1: Summary of classical optimization methods, including derivative types, search directions, and rules for step size.

Condition	Definition	Role / Notes
Armijo (Sufficient Decrease)	$f(x + \alpha d) \leq f(x) + c_1 \alpha g^T d$	Ensures steps are not too large; prevents divergence.
Wolfe Conditions	Armijo + curvature: $g(x + \alpha d)^T d \geq c_2 g^T d$	Prevents α from being too small; required for NCG global convergence.
Zoutendijk Condition	Angle condition: $\cos(\theta_k) = \frac{-g_k^T d_k}{\ g_k\ \ d_k\ } \geq \delta > 0$	Ensures descent direction quality; with Wolfe guarantees $\ g_k\ \rightarrow 0$.

Table 1.2: Global convergence conditions used in line-search algorithms (Armijo, Wolfe, Zoutendijk).

1.1 Exercise 1

1.1.1 Part a)

Zoutendijk Convergence Statement Let $(x_k)_{k \in \mathbb{N}_0} \subset \Omega \subset \mathbb{R}^n$ be the iterates of a descent method applied to a continuously differentiable function $f : \Omega \rightarrow \mathbb{R}$, bounded from below, whose gradient is Lipschitz continuous with constant $L > 0$. Assume the following:

- The step sizes $\alpha_k > 0$ satisfy the Wolfe conditions,
- The search directions satisfy $\nabla f(x_k)^T d_k < 0$,
- The angles θ_k between d_k and $-\nabla f(x_k)$ satisfy

$$\cos(\theta_k) = \frac{-\nabla f(x_k)^T d_k}{\|\nabla f(x_k)\| \|d_k\|} \geq \delta > 0. \quad (1.1)$$

We prove:

$$\lim_{k \rightarrow \infty} \|\nabla f(x_k)\| = 0.$$

Prove the decrease estimate We first show that there exists a constant $C > 0$ such that

$$f(x_{k+1}) \leq f(x_k) - C \cos(\theta_k)^2 \|\nabla f(x_k)\|^2. \quad (1.2)$$

Let $g_k := \nabla f(x_k)$. Because g is Lipschitz continuous with constant L , we have

$$\|g_{k+1} - g_k\| \leq L \|x_{k+1} - x_k\| = L \alpha_k \|d_k\|. \quad (1.3)$$

Taking inner products with d_k yields

$$|g_{k+1}^T d_k - g_k^T d_k| \leq L \alpha_k \|d_k\|^2. \quad (1.4)$$

The Wolfe curvature condition gives

$$g_{k+1}^T d_k \geq c_2 g_k^T d_k. \quad (1.5)$$

Combining (1.4) and (1.5),

$$c_2 g_k^T d_k \leq g_{k+1}^T d_k \quad (1.6)$$

$$\leq g_k^T d_k + L \alpha_k \|d_k\|^2. \quad (1.7)$$

Since $g_k^T d_k < 0$, we rearrange:

$$(1 - c_2)(-g_k^T d_k) \geq L \alpha_k \|d_k\|^2, \quad (1.8)$$

hence

$$\alpha_k \geq \frac{1 - c_2}{L} \frac{-g_k^T d_k}{\|d_k\|^2}. \quad (1.9)$$

Now apply the Armijo condition:

$$f(x_{k+1}) \leq f(x_k) + c_1 \alpha_k g_k^T d_k. \quad (1.10)$$

Substituting the lower bound (1.9):

$$f(x_{k+1}) \leq f(x_k) + c_1 \left(\frac{1 - c_2}{L} \frac{-g_k^T d_k}{\|d_k\|^2} \right) g_k^T d_k \quad (1.11)$$

$$= f(x_k) - \frac{c_1(1 - c_2)}{L} \frac{(g_k^T d_k)^2}{\|d_k\|^2}. \quad (1.12)$$

Using the identity

$$\cos(\theta_k) = \frac{-g_k^T d_k}{\|g_k\| \|d_k\|}, \quad (1.13)$$

we have

$$\frac{(g_k^T d_k)^2}{\|d_k\|^2} = \cos(\theta_k)^2 \|g_k\|^2. \quad (1.14)$$

Therefore,

$$f(x_{k+1}) \leq f(x_k) - \frac{c_1(1-c_2)}{L} \cos(\theta_k)^2 \|g_k\|^2. \quad (1.15)$$

Define

$$C := \frac{c_1(1-c_2)}{L} > 0, \quad (1.16)$$

and the claim (1.2) follows.

(b) Prove the series is finite

From (1.2),

$$C \cos(\theta_k)^2 \|g_k\|^2 \leq f(x_k) - f(x_{k+1}). \quad (1.17)$$

Summing from $k = 0$ to N ,

$$C \sum_{k=0}^N \cos(\theta_k)^2 \|g_k\|^2 \leq \sum_{k=0}^N (f(x_k) - f(x_{k+1})) \quad (1.18)$$

$$= f(x_0) - f(x_{N+1}). \quad (1.19)$$

Since f is bounded below by $f_{\inf}$,

$$f(x_0) - f(x_{N+1}) \leq f(x_0) - f_{\inf} < \infty. \quad (1.20)$$

Thus, for all N ,

$$\sum_{k=0}^N \cos(\theta_k)^2 \|g_k\|^2 \leq \frac{f(x_0) - f_{\inf}}{C}. \quad (1.21)$$

Taking $N \rightarrow \infty$,

$$\boxed{\sum_{k=0}^{\infty} \cos(\theta_k)^2 \|\nabla f(x_k)\|^2 < \infty}. \quad (1.22)$$

(c) Conclude the Zoutendijk convergence statement

From the angle condition (1.1),

$$\cos(\theta_k) \geq \delta > 0 \quad \text{for all } k. \quad (1.23)$$

Hence

$$\delta^2 \|g_k\|^2 \leq \cos(\theta_k)^2 \|g_k\|^2. \quad (1.24)$$

Using part (b),

$$\sum_{k=0}^{\infty} \delta^2 \|g_k\|^2 \leq \sum_{k=0}^{\infty} \cos(\theta_k)^2 \|g_k\|^2 < \infty.$$

Therefore,

$$\sum_{k=0}^{\infty} \|g_k\|^2 < \infty.$$

Since the summands are non-negative, a convergent sum implies

$$\|g_k\|^2 \rightarrow 0, \quad \text{hence } \boxed{\|\nabla f(x_k)\| \rightarrow 0}.$$

This proves the Zoutendijk convergence statement.

1.2 Exercise 2

1.2.1 Part a)

The linesearch with the Zoom algorithm consists of two phases. First, an *expansion phase* generates trial step sizes until an interval $[\beta_1, \beta_2]$ is found that is guaranteed to contain a step length satisfying the strong Wolfe conditions. Second, the *Zoom* procedure performs a bisectiontype search within this interval. At each midpoint, Armijo's sufficient decrease condition and the strong curvature condition are checked. If both hold, the step length is accepted. Otherwise, the interval is updated according to the sign of the directional derivative. This ensures that Zoom returns a step size satisfying the strong Wolfe conditions, which in turn allows the use of Zoutendijk's theorem and guarantees global convergence for descent methods.

Concise but explanatory description. The algorithm does not attempt to compute an optimal step size directly. Instead, the expansion phase builds a sequence $\alpha_0, \alpha_1, \alpha_2, \dots$ and tests each α_i . If Armijo fails or if $\nabla f(x + \alpha_i d)^T d \geq 0$, then the step size is known to lie in the interval $[\alpha_{i-1}, \alpha_i]$, which becomes the input to the Zoom routine.

The Zoom procedure then performs a safeguarded bisection. At each midpoint $\alpha = (\beta_1 + \beta_2)/2$ it checks Armijo and the strong curvature condition. If both hold, α is accepted. If Armijo fails, the upper bound is reduced. If Armijo holds but the curvature condition fails, the sign of $\nabla f(x + \alpha d)^T d$ determines which half of the interval can still contain a valid step. This process ensures that Zoom converges to a step size satisfying the strong Wolfe conditions.

Because the strong Wolfe conditions enforce both sufficient decrease and control over curvature, the assumptions of Zoutendijk's theorem are met. Consequently, for any descent direction, the gradients satisfy $\|\nabla f(x_k)\| \rightarrow 0$, establishing global convergence of the method.

1.2.2 Part b)

```

1
2 function alpha = zoom_wolfe(f, grad_f, x, d, alpha_init, c1, c2)
3     alpha0 = 0;
4     alpha1 = alpha_init;
5
6     i = 1;
7
8     while true
9         phi_alpha1 = f(x + alpha1 * d);
10        phi0 = f(x);
11        grad_phi0 = grad_f(x)' * d;
12
13        if (phi_alpha1 > phi0 + c1 * alpha1 * grad_phi0) || ...
14            (phi_alpha1 >= f(x + alpha0 * d) && i > 1)
15            alpha = zoom(alpha0, alpha1);
16            return;
17        end
18
19        grad_phi_alpha1 = grad_f(x + alpha1 * d)' * d;
20
21        if abs(grad_phi_alpha1) <= -c2 * grad_phi0
22            alpha = alpha1;
23            return;
24        end
25
26        if grad_phi_alpha1 >= 0
27            alpha = zoom(alpha1, alpha0);
28            return;
29        end
30
31        alpha0 = alpha1;

```

```
32     alpha1 = 2 * alpha1;
33     i = i + 1;
34 end
35
36 function a = zoom(beta1, beta2)
37     while true
38         a = 0.5 * (beta1 + beta2);
39         phi_a = f(x + a * d);
40
41         if (phi_a > phi0 + c1 * a * grad_phi0) || ...
42            (phi_a >= f(x + beta1 * d))
43             beta2 = a;
44         else
45             grad_phi_a = grad_f(x + a * d)' * d;
46             if abs(grad_phi_a) <= -c2 * grad_phi0
47                 return;
48             end
49             if grad_phi_a * (beta2 - beta1) >= 0
50                 beta2 = beta1;
51             end
52             beta1 = a;
53         end
54     end
55 end
56 end
```

Zoom wolfe

1.3 Exercise 3

1.3.1 Part c)

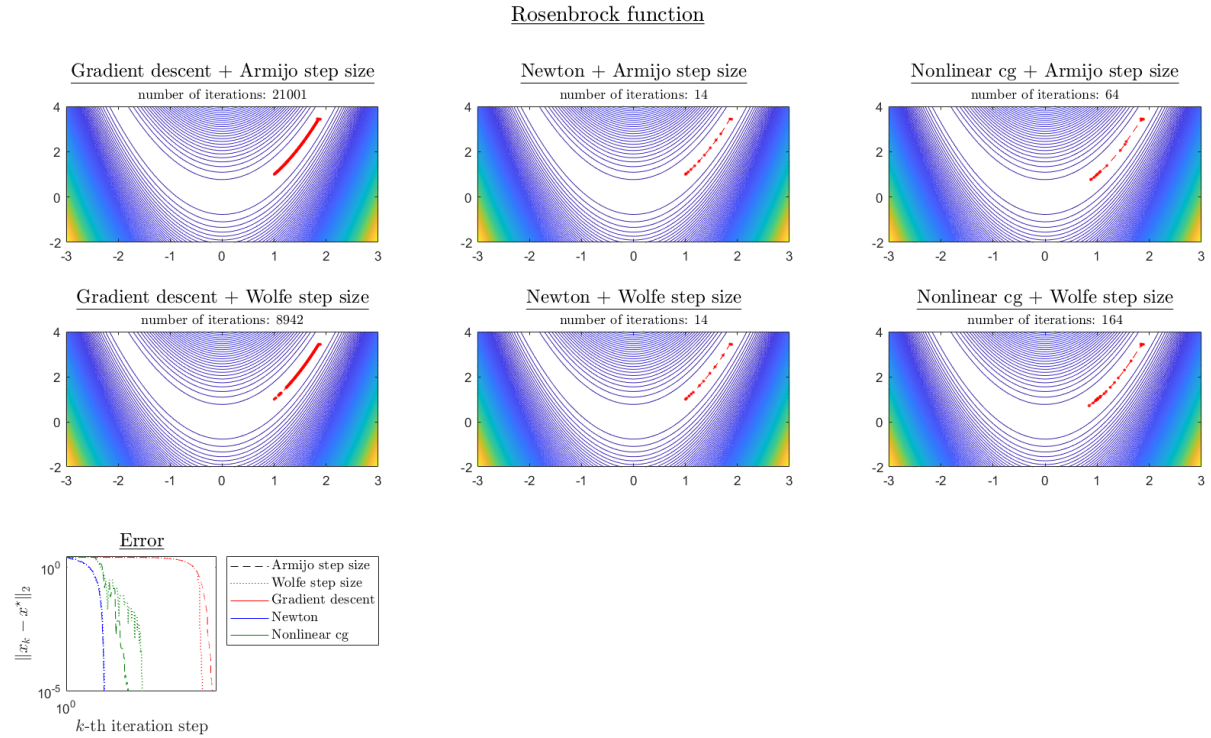


Figure 1.1: Zoomwolf Comparison

Summary

The plots compare three optimization methods (gradient descent, Newton, and nonlinear conjugate gradient) under two step-size strategies: Armijo backtracking and the Zoom algorithm enforcing the strong Wolfe conditions.

Gradient descent. With Armijo step sizes, gradient descent becomes extremely slow on the Rosenbrock valley because the Armijo rule tends to select very small step lengths. Using Zoom + Wolfe yields significantly larger and more effective steps, leading to fewer iterations, although the method remains slow.

Newton's method. Newton's method converges rapidly with either Armijo or Wolfe step sizes. Because the Newton direction already incorporates curvature information, the line-search has little influence on performance; Wolfe merely confirms that the unit step is acceptable.

Nonlinear conjugate gradient. This method is highly sensitive to the step-size rule. Armijo tends to produce steps that are too small, degrading the conjugacy of the search directions and slowing convergence. In contrast, the strong Wolfe conditions are known to preserve conjugacy, and the Zoom algorithm provides the required step lengths, resulting in a more stable iteration.

Overall. Armijo provides only sufficient decrease and often yields conservative step sizes. The Zoom algorithm, by enforcing both sufficient decrease and the strong curvature condition, avoids excessively short steps and guarantees the assumptions of Zoutendijk's theorem. The figure illustrates that Wolfe step sizes are essential for nonlinear CG, beneficial for gradient descent, and largely neutral for Newton's method.

Exercise sheet 7 for Numerical Optimization

Dr. Michael Lehn

Tobias Born

Deadline: December 3, 2025

Exercise 1 (Convergence statement (Zoutendijk) of descent methods, 9+9+9 points)

You already got a convergence statement of descent methods in the lecture. Now you will get to know a second one, the theorem of Zoutendijk. It demands more assumptions and yields classical convergence to a stationary point, not only that every accumulation point is a stationary point.

It states the following: Let $x_0 \in \mathbb{R}^n$ and $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be bounded from below. Let $(x_k)_{k \in \mathbb{N}_0} \subset \mathbb{R}^n$ be the iterates of a descent method and $\Omega \subset \mathbb{R}^n$ an open set in such a way that $(x_k)_{k \in \mathbb{N}_0} \subset \Omega$. Assume that f is continuously differentiable on Ω and that ∇f is Lipschitz continuous on Ω with Lipschitz constant $L > 0$. If

- the step sizes $\alpha_k > 0$ of all iteration steps fulfil the Wolfe condition,
- the directions $d_k \in \mathbb{R}^n$ of all iteration steps are search directions, so $\nabla f(x_k)^T d_k < 0$,
- and the angles θ_k between the search directions d_k and the gradients $\nabla f(x_k)$ are uniformly bounded away from 90° , meaning that there exists a $\delta > 0$ such that

$$\cos(\theta_k) = \frac{-\nabla f(x_k)^T d_k}{\|\nabla f(x_k)\| \|d_k\|} \geq \delta, \quad (1)$$

it holds true that

$$\lim_{k \rightarrow \infty} \|\nabla f(x_k)\| = 0.$$

- a) Prove that there is $C > 0$ such that

$$f(x_{k+1}) \leq f(x_k) - C \cos(\theta_k)^2 \|\nabla f(x_k)\|_2^2.$$

- b) Prove that

$$\sum_{k=0}^{\infty} \cos(\theta_k)^2 \|\nabla f(x_k)\|_2^2 < \infty.$$

- c) Use b) to prove the Zoutendijk convergence statement.

Exercise 2 (Zoom algorithm for strong Wolfe conditions, 9+9 points)

The Zoom algorithm is a common algorithm for determining a step size α that satisfies the strong Wolfe conditions for an iterate x and a search direction d . Remember that on sheet 5 exercise 3b you implemented an algorithm for finding a step size that satisfies the Armijo condition (the so-called *backtracking algorithm*). The Zoom algorithm is:

Algorithm 1: Zoom algorithm for strong Wolfe conditions

```
1 Input:  $f : \mathbb{R}^n \rightarrow \mathbb{R}$ ,  $x \in \mathbb{R}^n$ ,  $d \in \mathbb{R}^n$ ,  $\alpha_{\text{init}} > 0$ ,  $0 < c_1 < c_2 < 1$ 
2 Output: step size  $\alpha > 0$  satisfying the strong Wolfe conditions
3  $\alpha_0 \leftarrow 0$ 
4  $\alpha_1 \leftarrow \alpha_{\text{init}}$ 
5  $i \leftarrow 1$ 
6 while True do
7   if  $f(x + \alpha_i d) > f(x) + c_1 \alpha_i \nabla f(x)^T d \vee f(x + \alpha_i d) \geq f(x + \alpha_{i-1} d)$  then
8      $\text{return zoom}(\alpha_{i-1}, \alpha_i)$ 
9   if  $|\nabla f(x + \alpha_i d)^T d| \leq c_2 |\nabla f(x)^T d|$  then
10     $\text{return } \alpha_i$ 
11   if  $\nabla f(x + \alpha_i d)^T d \geq 0$  then
12     $\text{return zoom}(\alpha_i, \alpha_{i-1})$ 
13    $\alpha_{i+1} \leftarrow 2\alpha_i$ 
14    $i \leftarrow i + 1$ 
15
16 function  $\alpha = \text{zoom}(\beta_1, \beta_2)$ 
17 while True do
18    $\alpha \leftarrow \frac{\beta_1 + \beta_2}{2}$ 
19   if  $f(x + \alpha d) > f(x) + c_1 \alpha \nabla f(x)^T d \vee f(x + \alpha d) \geq f(x + \beta_1 d)$  then
20      $\beta_2 \leftarrow \alpha$ 
21   else
22     if  $|\nabla f(x + \alpha d)^T d| \leq c_2 |\nabla f(x)^T d|$  then
23        $\text{return } \alpha$ 
24     if  $\nabla f(x + \alpha d)^T d (\beta_2 - \beta_1) \geq 0$  then
25        $\beta_2 \leftarrow \beta_1$ 
26      $\beta_1 \leftarrow \alpha$ 
```

- a) Understand what the algorithm is doing. Since the algorithm is rather tricky, it is sufficient to understand its essentials.

Hint: 1) When calling $\text{zoom}(\beta_1, \beta_2)$, it is ensured that there is an interval with step sizes satisfying the strong Wolfe conditions between β_1 and β_2 . Make yourself clear why.

2) Make yourself clear that the inner function zoom is performing a bisection.

- b) Create a MATLAB programme `zoom_wolfe.m` that implements algorithm 1. It should be passed a function handle `f`, a function handle `grad_f`, an iterate $x \in \mathbb{R}^n$, a search direction $d \in \mathbb{R}^n$ and parameters $\alpha_{\text{init}} > 0$ and $0 < c_1 < c_2 < 1$.

The programme will be tested in the next exercise.

Exercise 3 (Comparison of descent methods and step size strategies using Rosenbrock's function, 5 points)

As grand finale to the descent methods, we will now compare all descent method variants we have looked at - gradient descent, Newton, nonlinear cg (Fletcher-Reeves) - using both step size strategies we have implemented, backtracking for an Armijo step size and the zoom algorithm for a strong Wolfe step size, using Rosenbrock's function.

Therefore, execute the MATLAB programme `comparison_descent_methods.m` and reflect on the results.